Session 12 Teamwork — Finish Making B4F Board's API Observable, Describable, and Controlled

Context

Work in your team (six teams of four students). CORE just gave Projects a Guard on `create` and `update` only, Swagger documentation on every existing route, and meaningful logs on create/delete. Projects' own DELETE, and everything on Workspaces, are still exactly as they were before class.

B4F Board's API must now be usable by another developer and controlled for internal write operations. This is a requirements/proof task, not a file-by-file recipe — you decide exactly which routes need protection and how to prove it, the same way you'd reason about it on a real team.

Goal

By the end, every real write route on both resources is protected, every rejection is proven to never reach the handler, meaningful logging exists wherever it's missing, the entire remaining API surface is documented through Swagger, and every existing MongoDB behavior still works exactly as before.

What to build

Your team must:

- **Identify** every write route (`POST` / `PATCH` / `DELETE`) on both resources that is not yet protected by `InternalToolGuard` — do not assume; check the real current state of both controllers.
- **Apply** `InternalToolGuard` to each one you identified, the same way it was already applied live to Projects' own `create` / `update` .
- **Prove**, for each newly-protected route, that a request without the real header returns `403` AND produces no log line at all from the owning service — rejection must happen before the handler runs, not merely return the right status code by coincidence.
- **Add meaningful logging** wherever a real mutation can succeed but currently logs nothing — reusing the exact Logger pattern already shown on Projects, at the layer where the real outcome is known (the service), not the controller.
- **Document** the entire remaining API surface through Swagger — every route that doesn't yet have a real `@ApiTags` / `@ApiOperation` , the header requirement on every newly-guarded route (without ever exposing the real key value), AND `FindWorkspacesQueryDto` 's own `name` / `slug` filter fields with `@ApiPropertyOptional` — the same standard CORE already applied live to `FindProjectsQueryDto` 's own `name` / `key` fields.
- **Use Swagger UI yourselves**, as another developer would, to confirm every route — old and new — is understandable and testable without reading the source code.
- **Preserve** all existing MongoDB CRUD/filtering behavior on both resources — re-run the real curl/mongosh checks from Session 11B's own opening review as your own regression proof.

Rules

Allowed: everything already shown live — `@UseGuards`, `@ApiHeader`, `@ApiTags`, `@ApiOperation`, `@ApiProperty` / `@ApiPropertyOptional`, Nest's own `Logger`, `ConfigService`. No new packages, no new decorators beyond what class already used. **Not yet:** no JWT, no users, no roles, no custom decorators, no interceptors, no database-level uniqueness constraints or indexes.

Test it (design your own full proof — this is a starting point, not the whole list)

BASH

```
curl -i -X DELETE localhost:3000/projects/1
curl -i -X POST localhost:3000/workspaces -H "Content-Type: application/json" -d
'{"name":"Team Test","slug":"team-test"}'
curl -i -X POST localhost:3000/workspaces -H "Content-Type: application/json" -H "x-
internal-tool-key: <your real local value>" -d '{"name":"Team Test","slug":"team-test"}'
curl -i -X PATCH localhost:3000/workspaces/<its id> -H "Content-Type: application/json" -d
'{"description":"no key"}'
curl -i "localhost:3000/workspaces?name=nova"
```

Watch the server log after each rejected request — if anything from `WorkspacesService` or `ProjectsService` appears, the handler ran, and the Guard is not actually protecting that route.

Done when

- Every write route on both resources — not just the two shown live — rejects without the header and succeeds with it.
- Every rejection you tested produced zero log output from the owning service.
- Every mutating route that can succeed logs a real, meaningful line when it does.
- `http://localhost:3000/docs` fully documents both resources — every route tagged, every handler summarized, every guarded route's header requirement stated without revealing its value, and `GET /workspaces`'s own filter parameters shown with real example values, same as `GET /projects`.
- `GET /workspaces` with/without filters, and every other previously-working route, still behaves exactly as it did before your team's changes — proven, not assumed.
- Every team member can explain, in their own words, why a Guard answers "is this request allowed," never "who is this."

Working time

About 45-50 minutes — reasoning about which routes need protection, applying a pattern you already watched built twice, and proving it rigorously, both yourselves.